

4_bubblesort

```
#include <iostream> // Used for input and output
#include <omp.h> // Used for OpenMP parallel programming
#include <ctime> // Used for measuring execution time
using namespace std; // Avoids writing std:: before cout, cin, etc.
// Function to perform parallel bubble sort
void bubblesort(int a[], int n)
{
    // Outer loop runs n times to make sure array becomes fully sorted
    for (int i = 0; i < n; i++)
    {
        // first decides whether this pass starts from index 0 or index 1
        // If i is even, first = 0 → compare (0,1), (2,3), (4,5)
        // If i is odd, first = 1 → compare (1,2), (3,4), (5,6)
        int first = i % 2;
        // OpenMP parallel for creates multiple threads for the loop below
        // shared(a, first) means all threads use the same array and first value
        #pragma omp parallel for shared(a, first)
        // This loop compares alternate pairs
        // j increases by 2 because we compare non-overlapping pairs
        for (int j = first; j < n - 1; j += 2)
        {
            // If current element is greater than next element, swap them
            if (a[j] > a[j + 1])
            {
                // Store next element temporarily
                int temp = a[j + 1];
                // Move current larger element to next position
                a[j + 1] = a[j];
                // Put smaller element at current position
                a[j] = temp;
            }
        }
    }
    // Print sorted array after sorting is completed
    cout << "Sorted list: (Bubble Sort)\n";
    // Loop to print all elements of sorted array
    for (int i = 0; i < n; i++)
    {
        cout << a[i] << " ";
    }
}
// Main function starts here
int main()
{
    // Number of elements in the array
```

```

int n = 100;
// Variables used for storing start and end time
clock_t t1, t2;
// Print total number of elements
cout << "Total number of elements: " << n << endl;
// Create array of size n
int a[n];
// Inform user that array is stored in descending order

cout << "Storing elements in descending order...\n\n";
// Fill array in descending order
// Example: if n = 100, array becomes 100, 99, 98, ...
for (int i = 0; i < n; i++)
{
    a[i] = n - i;
}
// Print original array before sorting
cout << "Actual list:\n";
// Loop to print original array
for (int i = 0; i < n; i++)
{
    cout << a[i] << " ";
}
cout << "\n";
// Store time before sorting starts
t1 = clock();
// Call bubble sort function
bubblesort(a, n);
// Store time after sorting ends
t2 = clock();
// Print execution time
// Difference between t2 and t1 gives time taken by sorting
printf("\nExecution Time: %f", (double)(t2 - t1) /
CLOCKS_PER_SEC);
// End of program
return 0;
}

```